

# Módulo de Prompt Engineering

## Sistema de Evaluación

En este módulo se plantean **dos ejercicios prácticos** y **un cuestionario final**, con el siguiente esquema de evaluación:

- **Ejercicio 1:** entrega obligatoria pero no evaluable
  - **Ejercicio 2 (Entregable final):** entrega obligatoria y es el **70% de la nota final**
  - **Cuestionario final:** obligatoria realización y es el **30% de la nota final**
- 

## Ejercicio 1: Entregable no evaluable

### Objetivo

Utilizar LLMs para extraer información de descripciones de alojamientos de AirBnB.

### Datasets

- `listings1_cleaned.csv`: Información de viviendas de AirBnB en Oporto
- `Airbnb_reviews_5000.csv`: Comentarios de usuarios sobre alojamientos

### Primera Parte – Extracción de entidades de descripciones

1. Filtrar el dataset con una función que seleccione las 10-20 reseñas con mayor longitud de texto en la columna "descripción" (las consideradas más relevantes)

2. Elaborar un prompt para que un LLM extraiga un JSON con las entidades:

- ``name``: Nombre del alojamiento
- ``location``: Ubicación
- ``main_characteristics``: Características principales
- ``type``: Tipo de alojamiento
- ``size``: Tamaño
- ``capacity``: Capacidad
- ``key_amenities``: Lista de comodidades clave
- ``proximity_highlights``: Lista de puntos de interés cercanos

3. Utilizar el proveedor LLM de tu elección (Gemini de Google o modelos de Groq)

## Segunda Parte – Análisis de comentarios

1. Realizar un análisis de los comentarios de los apartamentos a través de un LLM.
2. Extraer información relevante como: Limpieza, ubicación, relación calidad-precio, sentimiento del comentario u otros aspectos relevantes (formato de salida a tu elección)

## Ejercicio 2: Entregable Final

### Análisis de reseñas de videojuegos con LLMs

Este ejercicio constituye el **entregable principal del módulo** y tiene como objetivo aplicar técnicas de **Prompt Engineering** para extraer información valiosa a partir de texto no estructurado utilizando **Large Language Models (LLMs)**.

### Dataset

Se trabajará con un archivo CSV llamado: **videogames\_reviews.csv**

Cada fila del DataFrame representa una reseña de un videojuego y contiene las siguientes columnas:

- **Contenido:** texto libre de la reseña.
- **Valoración:** Recomendado o No recomendado.
- **Recomendado binario:** 1 si es *Recomendado*, 0 si es *No recomendado*.

Se trata de un dataset más **crudo** que el utilizado en el Ejercicio 1, por lo que requiere un mayor trabajo de preprocesado y diseño de prompts.

---

### Objetivo del ejercicio

El objetivo es diseñar un **pipeline en dos pasos con LLMs** que permita:

1. **Filtrar reseñas relevantes**, eliminando contenido poco informativo.
2. **Extraer información estructurada** de las reseñas relevantes para obtener insights útiles.

Este enfoque refleja un caso realista de uso de LLMs, donde se prioriza dividir tareas complejas en subtareas más simples y controlables.

---

## Pasos del ejercicio

### Paso 1: Selección inicial de reseñas

Se parte de un DataFrame filtrado previamente con:

- **Las 100 reseñas con mayor longitud de contenido.**

Este paso ya está proporcionado en el notebook y sirve para garantizar que el texto tenga suficiente información potencial.

---

### Paso 2: Filtrado de contenido relevante mediante Prompting

En este segundo paso deberás:

- Instruir a un **primer LLM** para que analice las reseñas.
- Filtrar únicamente aquellas filas cuyo contenido:
  - Aporte información relevante.
  - Permita extraer opiniones, aspectos positivos/negativos o insights útiles.

El resultado de este paso debe ser un **nuevo DataFrame reducido**, eliminando reseñas irrelevantes (por ejemplo: texto vacío, spam, comentarios sin contenido semántico, etc.).

**Clave del ejercicio:** el diseño del prompt y la estrategia de batching/tokenización.

---

### Paso 3: Extracción de información estructurada con un segundo LLM

A partir del DataFrame ya filtrado, se realizará una segunda llamada (o conjunto de llamadas) a un **segundo LLM**, cuyo objetivo será **extraer información estructurada** de cada reseña.

Un ejemplo de formato final podría ser:

```
{
  "Id": "{{id}}",
  "SentimientoGeneral": "Positivo | Negativo | Neutral",
  "AspectosPositivos": "[lista de aspectos positivos mencionados]",
  "AspectosNegativos": "[lista de aspectos negativos mencionados]",
  "Dificultad": "Demasiado Fácil | Fácil | Equilibrado | Difícil | Demasiado Difícil | No Mencionado",
  "Recomendado": "{{recomendado}}"
}
```

Este esquema es **orientativo**. El formato final es **libre**, siempre que:

- Se extraigan **al menos 3 entidades o atributos relevantes**.
  - La información esté bien estructurada y justificada.
- 

## Recursos adicionales

Se proporciona un archivo de ejemplo:  **analisis\_videojuegos\_resultados.csv**

Este archivo contiene un posible resultado final con **otras entidades extraídas** y un formato diferente. Puede utilizarse como inspiración.

---

## Consideraciones técnicas y de evaluación

Se valorará especialmente:

- **Diseño y claridad de los prompts.**
- Separación correcta de tareas entre distintos LLMs.
- Elección justificada del proveedor y modelo:
  - API (Groq, Gemini, OpenAI, etc.).
  - Modelos locales vía Hugging Face.
- Gestión de limitaciones:
  - Tokens máximos por llamada.
  - Rate limits.
  - Tamaño de los lotes de filas enviados al modelo por llamada
- Justificación de decisiones técnicas:
  - Uso de `time.sleep()` u otros métodos
  - Número de filas por batch.
  - Número de llamadas al modelo.

El foco del ejercicio **no está solo en el resultado**, sino en el **razonamiento y las decisiones de Prompt Engineering** que hay detrás.

---

## Entregable

El entregable deberá incluir:

- El notebook completo y ejecutable.
- Explicaciones claras de cada paso.
- Justificación de prompts, modelos y parámetros utilizados.
- El resultado final estructurado (DataFrame in-situ en notebook o CSV).
- O repositorio de Github al que me debéis dar acceso: [Github | Alonso](#)